Week 4 Exercises: REST APIs and Platform-Specific Development

Learning Objectives

After completing these exercises, you should be able to:

• Retrieve data from a REST API using fetch and async/await.
• Display remote JSON data in a React Native application.
• Handle loading, error and empty states.
• Filter retrieved data locally.
• Use remote data with navigation and a Details screen.
• Recognize basic platform-specific differences between Android and iOS.
• Use Platform.OS, Platform.select() and platform-specific modules.
• Test and verify application behaviour on different platforms.
• Use React Native Paper components for common application states and user interaction.

You may continue your application from Week 3 or create a new application.

You may use DummyJSON, another public REST API, or another suitable API related to your application.

Part 1: REST APIs

Answer briefly:

1. What is a REST API?
2. What is an HTTP GET request?
3. What is JSON?
4. What do fetch, async and await do?
5. What is the purpose of useEffect when retrieving data when a screen opens?
6. Why must applications handle loading and error states?
7. What is the difference between a network error and an HTTP error such as 404?

Part 2: Retrieve Data from a REST API

Continue developing your application from Week 3 or create a new application with a similar structure.

You can use the same API as in the lecture demo:

https://dummyjson.com/posts

You may also choose another suitable public REST API.

Create a screen that retrieves data from the API and displays it as a list.

Your application must:

• Make an HTTP GET request using fetch.
• Use async/await.
• Convert the response to JSON.
• Store the retrieved data in React state.
• Display multiple items using FlatList.
• Display meaningful information from each item.
• Use suitable React Native or React Native Paper components.

Before building the UI, examine the JSON returned by your API and identify the data you need.

For example, when using the DummyJSON posts API, identify:

• the array containing the posts
• id
• title
• body
• any other information you want to display

Do not simply display the raw JSON response. Present the information using appropriate UI components.

For example, each post could be displayed using a React Native Paper Card.

Part 3: Loading and Error States

Network requests take time and may fail. Extend your application so that the user can see what is happening during the request.

Your application should handle the following states:

Initial
Show an appropriate initial or empty view.

Loading
Display an ActivityIndicator or React Native Paper loading indicator.

Success
Display the retrieved data.

Error
Display a clear error message and provide a Retry button.

Use React state to manage the loading and error information.

While loading:

• Show a loading indicator.
• Do not display an error message.

If the request succeeds:

• Hide the loading indicator.
• Display the retrieved data.

If the request fails:

• Hide the loading indicator.
• Display an error message.
• Provide a Retry button.

Remember that fetch() does not automatically treat every HTTP error response as a JavaScript exception. Check the HTTP response before processing the data.

Test the Loading State

Do not only implement the loading state. Test that it actually works.

Temporarily use the DummyJSON delay parameter to make the response slower. For example:

https://dummyjson.com/posts?delay=3000

Verify that the loading indicator is visible before the data appears.

After testing, you may remove the artificial delay.

Test the Error State

Test your error handling deliberately.

For example:

• Temporarily use an incorrect API URL.
• Request an endpoint that returns an HTTP error.

Verify that:

• The application does not crash.
• The error message is displayed.
• The Retry button is available.

After testing, restore the correct API address.

Part 4: Filter the Retrieved Data

Add a simple search/filter feature to your list.

The filtering should happen locally after the data has already been retrieved from the API.

Do not make a new API request for every search.

For example, when displaying posts, allow the user to filter posts based on words appearing in the title or body.

You may use React Native Paper's Searchbar.

Your application should:

• Store the search text in React state.
• Filter the retrieved array using JavaScript.
• Display only matching items in FlatList.
• Update the results when the search text changes.
• Display an appropriate message if no items match the search.

For example, you can use JavaScript methods such as:

filter()

includes()

toLowerCase()

Think about the difference between these two operations:

Fetching retrieves data from a remote service.

Filtering processes data that is already available in your application.

Part 5: Details Screen

Add a Details screen for an item in your list.

When the user selects an item:

• Navigate to the Details screen.
• Pass the selected item or its ID using route parameters.
• Display more detailed information about the selected item.
• Allow the user to navigate back to the list.

Use suitable React Native Paper components such as Card, Chip, Avatar, Text or Appbar.

If your Week 3 application already contains a similar Details screen, you may adapt and reuse it.

Part 6: Platform-Specific Development

Most React Native code can be shared between Android and iOS. However, sometimes part of the appearance or behaviour of an application should be different depending on the platform.

6.1 Explore the Current Platform

Answer briefly:

1. What does Platform.OS return?
2. What is Platform.select() used for?
3. Why might Android and iOS require different UI or behaviour?
4. What are .android.js and .ios.js files used for?

Display the current platform somewhere in your application during development.

For example:

<Text> We're running on {Platform.OS} </Text>

Run the application on the platforms available to you and observe the value.

6.2 Add a Small Platform-Specific UI Difference

Use Platform.OS or Platform.select() to make one small part of your UI different depending on the platform.

For example:

• Different spacing or margins.
• Different styling.
• Different text or labels.
• Different icons.
• Different component behaviour.

The difference should be visible enough that you can verify that the correct platform-specific value is being used.

Do not create separate versions of the whole application.

6.3 Create a Platform-Specific Component

React Native can automatically select different component implementations based on the platform.

Create a small component with separate Android and iOS implementations.

For example:

components/

PlatformInfo.android.js
PlatformInfo.ios.js

Both files should export a component with the same name, but their appearance or content should be slightly different.

For example:

Android version:

• Display an Android icon or text.
• Use a Material-style Card.
• Display information specific to the Android version.

iOS version:

• Display Apple/iOS-related text or an icon.
• Use a different Card style or layout.
• Display information specific to the iOS version.

Import the component without including the platform extension:

import PlatformInfo from '../components/PlatformInfo';

Use the component normally:

<PlatformInfo />

React Native should automatically select the appropriate implementation for the current platform.

If you also run the application on the web, you may create:

PlatformInfo.web.js

Test your implementation and verify which component is selected.

If you cannot test both Android and iOS, explain what you expect to happen on the platform you cannot test.

Part 7: Explore One More Platform Difference

Find one additional example of something that can differ between Android and iOS.

For example:

• Native dialogs or action sheets.
• Shadows and elevation.
• Keyboard behaviour.
• Status bar behaviour.
• Permissions.
• Date/time selection.
• Navigation conventions.

You do not have to implement a large feature.

Briefly describe:

1. What is different?
2. Why might a React Native developer need to consider this difference?
3. How could React Native handle it?

Optional: Implement or experiment with the feature in your application.

Part 8: Reflection

Answer briefly:

1. How was retrieving REST API data different from using the local JSON data from Week 3?
2. Why is a loading state necessary when working with REST APIs?
3. How did you test that your loading state works?
4. How did you test that your error handling works?
5. What is the difference between fetching and filtering data?
6. How does the filtering functionality in your application work?
7. What platform-specific UI difference did you implement?
8. How does React Native select between .android.js and .ios.js modules?
9. How did you use AI-assisted development tools, if any? Describe one suggestion you tested or verified yourself.

Submission

Submit:

• Your application source code.
• A screenshot of the successfully retrieved data.
• A screenshot showing the loading or error state.
• A screenshot showing the filtering functionality.
• A screenshot of the Details screen.
• A screenshot showing your platform-specific implementation.
• Your written answers.

Minimum Requirements

Your application must include:

• REST API data retrieved using fetch
• async/await
• JSON response handling
• React state for API data
• useEffect
• FlatList
• loading state
• error state
• Retry functionality
• tested loading and error behaviour

- local search/filtering
- empty search result handling
- Details screen and navigation
- appropriate React Native or React Native Paper UI components
- Platform.OS or Platform.select()
- one platform-specific component using .android.js and .ios.js